VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY

THE INTERNATIONAL UNIVERSITY

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



DESIGN AND EVALUATION OF A DUAL-MODE MOBILE FITNESS TRACKING APPLICATION WITH RULE-BASED ENVIRONMENT CLASSIFICATION

By

Nguyen Duc Bao – ITITIU22238

A pre-thesis submitted to the School of Computer Science and Engineering

in partial fulfillment of the requirements for the degree of

Bachelor of Information Technology

Ho Chi Minh City, Vietnam

2026

APPROVED BY:

_____,

Vo Thi Luu Phuong, Assoc. Prof.
Supervisor

_____
(Typed Committee name here)

_____
(Typed Committee name here)

_____
(Typed Committee name here)

_____
(Typed Committee name here)

PRE-THESIS COMMITTEE

(Whichever applies)

Nguyen Duc Bao

Acknowledgements

I would like to express my sincere gratitude to my supervisor and lecturers at the School of Computer Science and Engineering, International University, Vietnam National University Ho Chi Minh City, for their guidance, feedback, and academic support throughout the development of this project. Their comments helped me clarify the project scope, improve the system design, and complete this report in a more structured manner.

I would also like to thank my family for their constant support and encouragement during my studies. Their support gave me the motivation to continue improving my knowledge and complete this project.

Finally, I would like to thank my friends and classmates who provided feedback during the development and testing process. Their suggestions helped me identify practical problems in the application workflow, especially in workout tracking, data persistence, and user interface consistency.

This report is the result of my effort to design and implement a smartphone-based fitness tracking application that combines mobile development, sensor-based tracking, local storage, cloud synchronization, and data visualization.

Contents

Approval **1**

Acknowledgements **2**

List of Figures **6**

List of Tables **7**

Abstract **8**

1 Introduction **9**

1.1 Background 9

1.2 Problem Statement 9

1.3 Scope and Objectives 10

1.4 Assumptions and Constraints 11

1.5 Proposed Solution 12

1.6 Structure of the Thesis 12

2 Literature Review **13**

2.1 Overview 13

2.2 Mobile Fitness Tracking Applications 13

2.3 Outdoor Activity Tracking Using GPS/GNSS 13

2.3.1 GPS/GNSS as the Primary Outdoor Source 13

2.3.2 GPS Accuracy Limitations 14

2.3.3 Distance Calculation: the Haversine Formula 14

2.4 GPS Noise Handling 15

2.4.1 Validation Gates 15

2.4.2 Kalman Filtering 15

2.5 Indoor Activity Tracking Using Smartphone Sensors 18

2.5.1 Limitation of GPS Indoors 18

2.5.2 Environment Classification: Rule-Based vs. Learning-Based Approaches . . . 18
2.5.3 Step Counting: Platform Sensor vs. Raw Accelerometer . . . 18
2.5.4 Stride-Length-Based Indoor Distance Estimation . . . 19
2.6 Workout Metric Estimation . . . 20
2.6.1 Duration, Speed, and Pace . . . 20
2.6.2 Calorie Estimation . . . 20
2.7 Offline-First Persistence and Cloud Synchronisation . . . 21
2.8 State Management and Application Architecture . . . 21
2.9 Commercial Fitness Application Comparison . . . 22
2.10 Research Gap . . . 23

3 System Design and Implementation 24
3.1 System Overview and Architecture . . . 24
3.2 Database Schema . . . 26
3.2.1 Cloud Database Schema (Supabase) . . . 28
3.3 Environment Classifier . . . 28
3.4 Outdoor GPS Tracking Pipeline . . . 33
3.4.1 Distance Calculation . . . 34
3.4.2 Pace and Speed Calculation . . . 35
3.4.3 Route Smoothing and Display Cleanup . . . 35
3.5 Indoor Step Tracking Pipeline . . . 35
3.6 Calorie Estimation . . . 37
3.7 Synchronisation Strategy . . . 38
3.8 Backend Processing Pipeline . . . 38
3.9 Cross-Screen Data Consistency . . . 39
3.10 Workout Recording UX Flow . . . 39

4 Experiment Results and Evaluation 41
4.1 Comparison with Reference Applications . . . 41
4.1.1 GPS Distance Benchmark using MAPE . . . 42
4.2 Developer Walkthrough Review . . . 43
4.3 Software Verification . . . 44

4.4 Environment Classification Field Observation 44
4.5 Indoor Step Counting Evaluation 44
4.6 Evaluation Summary 45

5 Conclusion and Future Work **47**
5.1 Conclusion 47
5.2 Limitations 47
5.3 Future Work 48

Bibliography **50**

List of Figures

1.1 Workout tracking pipeline 12
3.1 System-level end-to-end workout data flow 25
3.2 Aetron system architecture 26
3.3 Cloud database ERD 29
3.4 Environment classifier decision logic 31
3.5 Tracking-engine detail: GPS-primary pipeline with step-count fallback . 32
3.6 Workout UX flow 40
4.1 Route visualisation comparison across applications 42
4.2 Initial MAPE benchmark validation 43

List of Tables

2.1 Kalman theory vs. confidence-blending implementation 17
2.2 Commercial fitness applications comparison 22
3.1 Isar `LocalWorkout` schema 27
3.2 Isar `LocalGPSPoint` schema 27
3.3 Environment classifier thresholds 30
3.4 GPS validation parameters 33
4.1 Metrics comparison across applications 41
4.2 Indoor step-counting accuracy 45
4.3 Evaluation results summary 46

Abstract

Smartphone fitness tracking has become a widely adopted technology for monitoring physical activity, with commercial platforms such as Strava, Nike Run Club, and Google Fit demonstrating the practical value of GPS-based outdoor tracking and pedometer-based indoor tracking. However, these platforms rely on proprietary, undisclosed algorithms. Existing open-source and student-level implementations typically address only isolated sub-problems—GPS recording, or step counting, or UI layout—without integrating environment classification, GPS validation, offline persistence, and cross-screen data consistency into a single documented, reproducible system. Furthermore, no academic prototype has openly combined a rule-based environment classifier with a local-first architecture and a cloud synchronisation layer in a way that supports future supervised learning extensions using real collected workout data.

This project addresses these gaps by designing and implementing *Aetron*, a Flutter-based fitness application that coordinates GNSS and pedometer sensors through a transparent, auditable pipeline. A deterministic rule-based *Environment Classifier* automatically selects between an outdoor GPS pipeline and an indoor step-counting pipeline based on a rolling window of sensor signals. Validated GPS fixes are accumulated via the Haversine formula for outdoor sessions, while indoor distance is estimated from pedometer readings scaled by user-specific stride-length coefficients. Completed sessions are persisted locally before synchronisation to a Supabase cloud backend, and a Clean Architecture separation ensures all screens share a single repository source of truth, guaranteeing cross-screen metric consistency.

Evaluation against five commercial reference applications on an identical outdoor route shows that Aetron recorded 0.52 km, within the 0.51–0.56 km range of its peers, with a separate MAPE benchmark on a pre-measured 800 m route yielding 2.5% error—within the "good" classification by established sport-science criteria. Indoor step-count trials produced a mean absolute error of 3.0%, below the 5% acceptance threshold, all ten force-close sessions were recovered intact, and all four project objectives are satisfied at the pre-thesis stage, with backend post-processing validation deferred to the full thesis.

Chapter 1

Introduction

1.1 Background

Smartphone-based fitness applications have become an essential tool for helping individuals maintain physical activity habits. Modern smartphones possess all the necessary hardware components, such as GPS/GNSS receivers and inertial sensors, to record and process workout data without external accessories.

While commercial platforms such as Strava and Nike Run Club demonstrate what a polished fitness application can achieve, their tracking algorithms are proprietary. Building a reliable fitness application from scratch in an academic context requires integrating several technical domains: satellite-based positioning, sensor data processing, real-time state management, and reliable data persistence. Coordinating these components into a consistent end-to-end workflow is the central engineering challenge of this project.

The application is named *Aetron*. It is implemented in Flutter [1], a cross-platform mobile framework that produces natively compiled applications from a single Dart codebase. Outdoor tracking uses the `geolocator` package for GNSS location streams [2]. Indoor tracking uses the `pedometer` package, which provides access to the device step-counter sensor [3]. State management is handled by Riverpod [4]. Local persistence uses Isar for workout and GPS-point records, SQLite via `sqflite` [5] for profile caching, and Supabase [6] for authentication, cloud synchronisation, storage, and server-side processing.

1.2 Problem Statement

Many smartphone fitness applications provide tracking features, but implementing those features correctly and consistently in a custom application remains technically demanding. The core difficulty is that workout tracking depends on multiple data sources that are individually unreliable: GPS coordinates may drift or jump due to signal multipath and satellite geometry [7]; step-counter readings vary by device and walking style [8]; and background services can be interrupted by the operating system during long workouts.

From a system perspective, the problem addressed by this project is:

> Design and implement a mobile fitness tracking application that correctly records indoor and outdoor workout sessions, calculates consistent real-time metrics, stores workout history reliably, synchronises account-level data to the cloud, and presents coherent summaries across the Home, Workout, Statistics, Calendar, and Profile modules—even when individual sensor readings are noisy or temporarily unavailable.

The following specific issues must be addressed:

- **Sensor reliability:** the application must decide whether GPS or step counting is appropriate while handling GPS drift, weak signal, step-counter variation, and temporary sensor gaps.
- **Metric consistency:** distance, duration, pace, steps, and calories must update during a session and remain coherent when the workout is completed.
- **Persistence and synchronisation:** completed workouts must survive restart, OS termination, logout, and temporary network loss before being synchronised to the cloud.
- **Cross-screen consistency:** the same saved workout record must appear identically in History, Statistics, Calendar, Summary, and Details views.

1.3 Scope and Objectives

This project targets individual users who wish to record walking, running, and indoor exercise sessions using only their smartphone. It is scoped as a student-level software engineering prototype evaluated against academic criteria, not as a medical-grade measurement device or a commercial product.

The scope includes: Flutter mobile application; user authentication and profile management; outdoor GPS tracking with validation and display smoothing; indoor step-based tracking; real-time metric calculation; local offline persistence; cloud synchronisation via Supabase; goal tracking; and Statistics and Calendar modules. The scope explicitly excludes: wearable device integration and heart-rate monitoring (which would enable more accurate calorie estimation but require additional hardware); production-grade route map-matching and routing intelligence beyond the prototype-level correction worker described in Section 3.8; and multi-user or social features. Calorie values are provided as estimates and are not intended for medical or clinical use.

The main objectives of the project are:

- **O1 — Adaptive Tracking Pipeline:** automatically classify the workout environment (outdoor or indoor) and route sensor data through the appropriate pipeline—validated GNSS distance calculation for outdoor sessions and step-based distance estimation for indoor sessions—with outdoor results falling within the range of established commercial applications under the same conditions.
- **O2 — Real-Time Session Experience:** display live workout metrics (time, distance, pace, steps, and calories) with updates no slower than once per second throughout the session.
- **O3 — Reliable Persistence and Cloud Synchronisation:** commit completed sessions to local storage before any network operation so that records survive restart, OS termination, or connectivity loss, and synchronise user profile and workout data with Supabase after authentication.
- **O4 — Cross-Screen Consistency and Modular Architecture:** ensure that the same saved workout record appears identically across History, Statistics, and Calendar screens, supported by a Clean Architecture codebase with clearly separated presentation, domain, data, and service responsibilities.

1.4 Assumptions and Constraints

The project operates under the following assumptions. Users are expected to grant location and activity-recognition permissions; without these, outdoor and indoor tracking cannot function. GNSS accuracy is assumed to be higher outdoors than indoors, consistent with documented smartphone positioning behaviour [7]. Indoor distance is treated as an estimate because the system does not use a calibrated foot-pod or treadmill interface. Calorie values are treated as approximations because accurate energy expenditure depends on physiological variables—including heart rate, running economy, and metabolic rate—that a smartphone cannot directly measure [9]. The system prioritises local persistence before cloud upload: a workout is committed to the local Isar store before any network operation is attempted, so a completed session is never lost due to connectivity failure.

Hardware and OS constraints include: GPS accuracy variability by device, satellite geometry, and urban canopy; step-counter availability differences between Android and iOS; operating-system battery-optimisation policies that may suspend background location services; and the Supabase free-tier rate limits on API calls and database size.

1.5 Proposed Solution

Aetron uses a layered, offline-first architecture designed to operate reliably regardless of network conditions. At the sensor tier, a deterministic, rule-based *environment classifier* inspects incoming GNSS and step-counter signals in real time to automatically select the appropriate tracking mode.

Depending on the classified environment, data flows through either a GPS pipeline or a step-based pipeline to calculate distance, pace, and estimated calorie expenditure. These computed metrics continuously update the application state to drive live UI re-renders. Upon completion, the session is persisted to local storage before being queued for cloud synchronisation. This high-level data flow is illustrated in Figure 1.1.

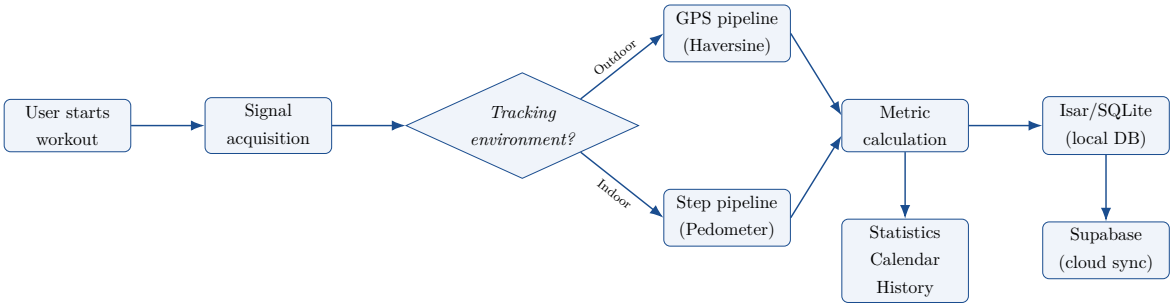


Figure 1.1: Workout tracking data processing pipeline.

1.6 Structure of the Thesis

This thesis is organised into five chapters:

- **Chapter 1** introduces the project background, problem statement, scope, objectives, assumptions, constraints, and proposed solution.
- **Chapter 2** reviews the technical foundations of smartphone fitness tracking, including GNSS tracking, indoor step counting, metric estimation, persistence, synchronisation, and related commercial applications.
- **Chapter 3** presents the design and implementation of Aetron, including the application architecture, local and cloud data model, tracking pipeline, backend processing, and user-facing modules.
- **Chapter 4** evaluates the prototype through comparison with selected reference applications, user-interface review, and software verification tests.
- **Chapter 5** concludes the project, identifies limitations, and proposes future improvements.

Chapter 2

Literature Review

2.1 Overview

This chapter reviews the technical foundations underpinning Aetron's design. The review focuses strictly on smartphone-only fitness tracking, which must derive all data from built-in GNSS receivers, inertial sensors, and application-level processing rather than dedicated external hardware. This constraint creates distinct algorithmic and architectural trade-offs that define the project's technical scope.

2.2 Mobile Fitness Tracking Applications

Mobile fitness applications transform raw smartphone sensor data into understandable workout metrics, allowing users to monitor progress toward health targets [10]. Building these applications primarily presents a system integration challenge: maintaining data consistency across sensor reading, local persistence, cloud synchronisation, and UI visualisation without relying on external hardware.

2.3 Outdoor Activity Tracking Using GPS/GNSS

2.3.1 GPS/GNSS as the Primary Outdoor Source

For outdoor activities, smartphones obtain position estimates from Global Navigation Satellite System (GNSS) receivers. In Flutter development, the `geolocator` package exposes these position fixes as a Dart stream, providing latitude, longitude, altitude, speed, heading, and a horizontal accuracy estimate in metres [2]. The accuracy estimate is key: it is the primary signal used by the environment classifier (Section 3.3) to decide whether the GPS fix is trustworthy enough to contribute to the distance accumulator.

A typical GPS-based workout pipeline follows this sequence: (1) request location permission; (2) start a continuous location stream; (3) receive position fixes with metadata; (4) validate each fix against accuracy, displacement, and speed criteria; (5) compute and accumulate the incremental Haversine distance between consecutive valid

fixes; (6) update real-time UI state; and (7) save the completed route and session summary.

2.3.2 GPS Accuracy Limitations

Smartphone GPS accuracy is affected by satellite geometry, signal multipath from buildings, and hardware quality [7]. While optimal conditions can yield 1.75 m accuracy [11], high-sensitivity GPS phones degrade significantly in dense urban canyons where multipath reflections dominate [12].

For a fitness application, these errors accumulate across thousands of position fixes. Common manifestations include stationary **drift**, sudden **multipath jumps**, **temporary signal loss** under tree cover or indoors, and mathematically impossible **speed spikes**.

These characteristics motivate the validation pipeline described in Chapter 3: raw fixes should not be accumulated directly without at least an accuracy gate and a speed sanity check.

2.3.3 Distance Calculation: the Haversine Formula

Given consecutive validated position fixes, the straight-line surface distance between them is computed using the Haversine formula [13]. For two geographic coordinates (ϕ_1, λ_1) and (ϕ_2, λ_2) expressed in radians:

$$a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos\phi_1 \, \cos\phi_2 \, \sin^2\left(\frac{\Delta\lambda}{2}\right) \tag{2.1}$$

$$c = 2 \, \operatorname{atan2}\left(\sqrt{a}, \, \sqrt{1-a}\right) \tag{2.2}$$

$$d = R \cdot c \tag{2.3}$$

where $R \approx 6{,}371\,\text{km}$ is the mean Earth radius, $\Delta\phi = \phi_2 - \phi_1$, and $\Delta\lambda = \lambda_2 - \lambda_1$. The Haversine formula is preferred over simpler planar approximations because it correctly handles the spherical geometry of Earth, avoiding significant errors at large inter-point distances or high latitudes. For the short segments between consecutive GPS fixes in a workout ($\ll 1\,\text{km}$), the difference from a planar approximation is negligible, but the formula is used for correctness.

Total session distance is the sum of all accepted segment distances:

$$D_{\text{total}} = \sum_{i=1}^{n-1} d(p_i,\, p_{i+1}) \tag{2.4}$$

where p_i and p_{i+1} are consecutive validated route points. The Dart implementation of this formula, delegating to the `geolocator` package's `distanceBetween` function (which internally implements Haversine), is shown in Listing 3.1 in Section 3.4.

2.4 GPS Noise Handling

2.4.1 Validation Gates

The simplest and most computationally efficient approach to GPS noise reduction is threshold-based outlier rejection. Before a position fix is accepted, it is compared against:

- An **accuracy gate**: fixes whose reported horizontal accuracy exceeds a threshold (e.g. 25 m) are discarded, as they are unlikely to represent the user's true position.
- A **minimum displacement threshold**: fixes that imply movement below a few metres since the last accepted fix are likely GPS jitter rather than real motion and can be skipped.
- A **speed sanity check**: fixes implying a ground speed above a physical maximum for the selected activity type (e.g. 12 m/s for walking, 25 m/s for cycling) indicate a data error and are rejected.

These rules are deterministic, require no training data, and add negligible computational overhead. Their thresholds are tunable and must be validated empirically for each activity type during the testing phase [14].

2.4.2 Kalman Filtering

For smoother position estimates, Kalman filtering [15] provides a principled framework for recursively combining a noisy measurement with a prior state estimate. In a one-dimensional application to GPS latitude or longitude, the filter maintains a state estimate $\hat{x}_k$ and an associated uncertainty P_k, which are updated at each time step k according to:

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k\big(z_k - \hat{x}_{k|k-1}\big), \quad K_k = \frac{P_{k|k-1}}{P_{k|k-1} + R} \tag{2.5}$$

where z_k is the raw GPS measurement, R is the measurement noise variance (derived from the reported accuracy), and K_k is the Kalman gain. When R is large (low-quality fix), K_k shrinks and the filter trusts the prior more; when R is small (high-quality fix), K_k approaches 1 and the measurement dominates. This self-weighting behaviour is the key intuition borrowed from Kalman theory.

Although the full Kalman recursion (with the predict-update cycle for P_k) is a standard reference technique for GPS smoothing [15], the current Aetron prototype replaces it with a lighter-weight **confidence-based blending** stage that preserves the same core intuition: GPS accuracy controls how much the new measurement is trusted. Instead of propagating a covariance matrix P_k, the implementation maps the reported `accuracy` value from the `geolocator` package to one of three discrete confidence levels and then selects a fixed blending weight α, which plays the same role as the Kalman gain K_k:

Listing 2.1: Confidence classification and position blending in `GpsSmoothingService` (`gps_smoothing_service.dart`)

```
// Step 1: map GPS accuracy to a confidence level
// Analogous to computing K_k from R in Kalman theory:
//   accuracy <= 15 m  (small R)  -> high confidence
//   accuracy <= 30 m  (medium R) -> medium confidence
//   accuracy >  30 m  (large R)  -> low confidence
GpsConfidence _confidenceFor({
  required double accuracyMeters,
  required double speedMs,
  required double timeDeltaSec,
  required double gpsGapDurationSec,
}) {
  if (gpsGapDurationSec > 5.0 || accuracyMeters > 30.0) {
    return GpsConfidence.low;
  }
  if (accuracyMeters > 15.0 || timeDeltaSec > 2.5 || speedMs
      <= 0.4) {
    return GpsConfidence.medium;
  }
  return GpsConfidence.high;
}

// Step 2: apply blending weight (alpha replaces K_k)
// smoothed = prev*(1 - alpha) + accepted*alpha
// High accuracy  -> alpha = 0.72  (trust measurement more,
```

```
    like large K_k)
// Medium accuracy -> alpha = 0.52
// Low accuracy    -> alpha = 0.34  (trust prior more, like
    small K_k)
var alpha = switch (confidence) {
  GpsConfidence.high   => 0.72,
  GpsConfidence.medium => 0.52,
  GpsConfidence.low    => 0.34,
};
// Result: smoothed coordinate
return LatLng(
  previousSmoothedPoint.latitude  * (1 - alpha) +
     acceptedPoint.latitude  * alpha,
  previousSmoothedPoint.longitude * (1 - alpha) +
     acceptedPoint.longitude * alpha,
);
```

Table 2.1 summarises the conceptual correspondence between the Kalman formulation and the prototype implementation.

Table 2.1: Correspondence between Kalman filter concepts and the prototype's confidence-based blending

Kalman concept	**Prototype equivalent**
Measurement noise R	`accuracy` (metres) from `geolocator`
Kalman gain K_k	Blending weight $\alpha \in \{0.34, 0.52, 0.72\}$
Uncertainty P_k	Not explicitly tracked; subsumed into discrete confidence level
Low R (good fix) $\Rightarrow$ high K_k	Small `accuracy` value ($\leq$15 m) $\Rightarrow$ `GpsConfidence.high` $\Rightarrow$ $\alpha = 0.72$
High R (bad fix) $\Rightarrow$ low K_k	Large `accuracy` value (>30 m) $\Rightarrow$ `GpsConfidence.low` $\Rightarrow$ $\alpha = 0.34$

This deterministic approach was chosen because it requires no training data, produces human-readable threshold decisions, and adds no inference latency on low-end devices [14]. The discrete confidence levels also make it straightforward to add bearing-aware spike correction and stationary-point freezing on top of the basic blending step.

2.5 Indoor Activity Tracking Using Smartphone Sensors

2.5.1 Limitation of GPS Indoors

Satellite-based positioning is unreliable or unavailable inside buildings because the receiver cannot maintain line-of-sight to enough satellites. This makes GPS the wrong sensor for treadmill workouts, gym sessions, and indoor walking activities. Harle's survey of indoor inertial positioning systems [14] confirms that Pedestrian Dead Reckoning (PDR) based on inertial sensors is the dominant alternative for indoor smartphone tracking, and step counting is the simplest PDR approach that does not require a dedicated IMU.

2.5.2 Environment Classification: Rule-Based vs. Learning-Based Approaches

The decision of whether to use GPS or step counting is called *environment classification*. While machine-learning approaches (e.g., CNNs or LSTMs) can classify complex activities with high accuracy [16, 17], they require curated datasets and introduce computational overhead.

For the pre-thesis prototype, Aetron implements a deterministic rule-based classifier that requires no training data and produces human-readable boundaries [14]. It evaluates a rolling window of the ten most recent GPS fixes against five signals: mean accuracy, displacement, speed, jitter ratio, and step-count delta. The architecture is designed to accommodate a learned classifier as a future extension once real-world sensor logs are collected.

2.5.3 Step Counting: Platform Sensor vs. Raw Accelerometer

Two implementation approaches are available for smartphone step counting. The first processes raw accelerometer data using custom peak-detection algorithms. This gives the developer full control and works on any device, but requires tuning for different gait patterns, phone placements, and walking speeds.

The second approach uses the operating system's built-in step-counter sensor when available. On Android, the `TYPE_STEP_COUNTER` sensor reports a monotonically increasing cumulative step count since the last device reboot [18]. This is the approach adopted in Aetron via the `pedometer` package [3]. The OS sensor is preferred because it

is managed at the hardware level on many devices (using a dedicated co-processor), which reduces battery consumption compared with continuously reading the accelerometer in software. Brajdic and Harle [8] validated that OS-level step counting on unconstrained smartphones achieves error rates well below 5% under typical walking conditions, supporting its suitability as the indoor tracking source.

2.5.4 Stride-Length-Based Indoor Distance Estimation

Step count alone does not yield distance. The relationship between steps and distance is:

$$D_{\text{indoor}} = N_{\text{steps}} \times L_{\text{stride}} \tag{2.6}$$

where N_{steps} is the step count accumulated during the session and L_{stride} is the estimated stride length. Gender-specific default values derived from anthropometric studies provide a reasonable starting point. Taborri et al. [19] report that average stride length scales approximately linearly with height. Experimental results from IMU-based stride length estimation indicate that errors can be reduced to approximately 2% using consumer-grade sensors [20], which supports using height-based stride coefficients as a practical smartphone-only approximation. Aetron implements separate walking and running coefficients so that the same step-count pipeline can support indoor walking and running sessions:

$$L_{\text{stride}} = \begin{cases} 0.43 \times H & \text{if walking, male} \\ 0.40 \times H & \text{if walking, female} \\ 0.67 \times H & \text{if running, male} \\ 0.62 \times H & \text{if running, female} \end{cases} \tag{2.7}$$

where H is the user's height in metres, read from the profile. The walking stride coefficients ($0.43H$ for males, $0.40H$ for females) are adapted from the height-scaling relationships reported by Taborri et al. [19] and validated in IMU-based stride estimation work [20]. The running coefficients ($0.67H$ for males, $0.62H$ for females) are prototype defaults; the implied run-to-walk stride ratio ($0.67H/0.43H \approx 1.56$) falls within the 50–60% increase commonly cited in biomechanics literature for comfortable running speeds [19, 21]. These height-scaled coefficients represent population averages; individual calibration against a known indoor distance would replace these defaults and reduce error for users whose stride length deviates significantly from the mean. If the user's height is unavailable, the application falls back to fixed defaults of 0.73 m/0.68 m for male/female

walking and 0.93 m/0.85 m for male/female running. The Dart implementation of these coefficients, including height-based computation and gender-specific fallback values, is shown in Listing 3.2 in Section 3.5.

2.6 Workout Metric Estimation

2.6.1 Duration, Speed, and Pace

Workout duration T is the elapsed wall-clock time between the session start and end timestamps. Average speed v_{avg} and pace p are derived from total distance D and duration:

$$v_{\text{avg}} = \frac{D}{T}, \qquad p = \frac{T}{D} \tag{2.8}$$

Pace is expressed as minutes per kilometre in the application UI. When the user pauses, elapsed pause time is excluded from T when computing moving pace, but is included in total duration displayed on the summary screen.

2.6.2 Calorie Estimation

Accurate energy expenditure measurement requires direct or indirect calorimetry equipment that is not available on consumer smartphones. Commercial fitness applications therefore rely on estimation models that combine user body metrics with activity data.

The Metabolic Equivalent of Task (MET) approach calculates energy expenditure as:

$$E_{\text{kcal}} = \text{MET} \times W_{\text{kg}} \times T_{\text{hours}} \tag{2.9}$$

where MET values represent activity-specific metabolic rates relative to resting metabolism [9]. However, accurate MET-based estimation requires activity classification, which introduces additional complexity and potential error.

Distance-based models provide a simpler alternative for ambulatory activities. General running calorie formulas estimate expenditure as approximately 0.63–0.75 kcal per kilogram per kilometre for moderate-pace running [22], though individual metabolic efficiency varies significantly.

Commercial applications use proprietary variants of these approaches but do not disclose their exact formulas. Strava's official documentation confirms that calorie calculations for runs use speed (grade-adjusted), weight, and moving time with an

undisclosed scaling factor [23]. Google Fit includes both activity expenditure and basal metabolic rate (BMR) in its calculations, requiring users to input height and weight [24, 25]. Nike Run Club and Samsung Health similarly combine body metrics (age, weight, height, gender) with activity data (distance, pace, duration) but do not publish the specific weighting or scaling factors applied [26, 27].

A 2025 systematic review and meta-analysis of 56 studies evaluating Apple Watch accuracy found that while heart rate measurements exhibited 4.43% error and step counts showed 8.17% error, calorie expenditure estimates demonstrated significantly higher error rates of approximately 27.96% across all activity types including walking, running, and cycling [28]. This pattern is consistent with broader reviews of consumer wearables reporting energy expenditure errors exceeding 30% across device brands [29]. This consistent inaccuracy exists even in devices with heart rate sensors and sophisticated proprietary algorithms.

Given this systematic limitation in commercial implementations and the absence of heart rate measurement capability in smartphone-only tracking, Aetron uses a simplified distance-based approximation described in Section 3.6. This deterministic approach prioritises cross-screen consistency and auditability over physiological precision. Calorie values are displayed as estimates and are not presented as medical-grade measurements.

2.7 Offline-First Persistence and Cloud Synchronisation

Workout data must survive application termination and network failure. This requires an offline-first architecture where sessions are written to a local database (e.g., Isar or SQLite [30]) immediately before any network operation. Once committed locally, the record is uploaded to a cloud backend (e.g., Supabase [6]) when network access becomes available, ensuring full functionality without connectivity.

2.8 State Management and Application Architecture

Robust state management and clear architectural separation are essential for maintaining UI consistency when handling asynchronous sensor streams. Aetron leverages Riverpod [4] for compile-safe reactive state management and Freezed [31] for immutable state classes. The codebase follows the Clean Architecture pattern [32], enforcing strict boundaries between the *presentation*, *domain*, and *data* layers.

2.9 Commercial Fitness Application Comparison

Commercial applications provide useful reference points for evaluating feature completeness and tracking design. Table 2.2 summarises publicly documented capabilities of selected platforms. Their internal algorithms are proprietary; the comparison is limited to published documentation, developer documentation, and publicly observable behaviour.

Table 2.2: Selected commercial fitness applications and their relevance to this project

Application	Documented tracking features	Calorie estimation	Relevance to Aetron
Strava [33]	Public Strava materials describe GPS-based activity recording, route visualisation, segment analysis, and pace charts.	Speed × weight × time + undisclosed scaling factor [23]	Reference for outdoor GPS tracking quality and route visualisation standards.
Nike Run Club [26]	NRC exposes indoor and outdoor run modes; indoor runs rely on device motion data instead of an outdoor GPS route.	BMR/BMI + weight + distance + pace (formula not disclosed)	Validates the indoor/outdoor dual-mode design and step-based distance approach.
Samsung Health [27]	Samsung Health developer documentation exposes step count, heart rate, sleep, and workout data, supporting on-device pedometry use cases.	Age + weight + height + gender + optional HR (formula not disclosed)	Reference for OS-level step counter integration and multi-activity tracking.
Google Fit [34]	Google Fit documentation provides sensor and recording APIs for step count, activity type, and derived distance from device sensors.	Activity + gender + height + weight + BMR [24, 25]	Reference for sensor integration and offline-first design patterns.

The comparison shows that dual-mode tracking (GPS for outdoor, step counting for indoor) is the established approach across all major platforms. The differentiating aspect of Aetron is the explicit, auditable rule-based environment classifier and the local-first architecture, both of which are described in detail in Chapter 3.

None of the surveyed commercial applications publicly disclose their complete calorie estimation formulas. Strava uses speed, weight, and time with an undisclosed scaling factor for runs, and power-based estimates for cycling [23]. Google Fit is the most transparent, documenting that it includes BMR in addition to activity expenditure and requires height and weight input [24, 25], but does not publish the specific calculation formula. Nike Run Club and Samsung Health request similar body metrics but provide no algorithmic documentation [26, 27].

Independent validation studies confirm that calorie estimation remains the least accurate metric in commercial fitness tracking. A 2025 meta-analysis covering 56 peer-reviewed studies found that consumer wearables achieve relatively low errors for heart rate (4.43%) and step count (8.17%), but calorie expenditure estimates exhibit systematic error of approximately 28% across all activity types [28]. This high error rate persists even in devices with heart rate sensors, accelerometers, and proprietary machine-learning models, highlighting the fundamental difficulty of estimating individual metabolic cost without direct calorimetry equipment.

This documented inaccuracy in commercial implementations, combined with the proprietary nature of their algorithms, motivates the use of a transparent, deterministic distance-based model in Aetron. The approach adopted in this project sacrifices potential physiological precision for consistency, auditability, and reproducibility—qualities that are important in an academic prototype where algorithmic decisions must be documented and justified.

2.10 Research Gap

The review in Sections 2.3–2.9 identifies three distinct gaps that this project addresses.

First, existing student and open-source fitness projects documented in the literature typically address only a subset of the end-to-end tracking problem. Tutorial-style Flutter projects (widely available on GitHub) focus on UI layout and Google Maps rendering but omit GPS validation, indoor fallback, and offline persistence. Open-source trackers such as OpenTracks implement outdoor GPS recording and GPX export but do not include environment classification or cross-screen metric consistency. Neither category provides a documented, reproducible description of the decision logic used to accept or reject GPS fixes.

Second, while commercial applications (Section 2.9) implement dual-mode tracking as a de facto standard, their environment classification and GPS validation pipelines are proprietary and cannot be studied, replicated, or compared by independent researchers. This opacity limits what academic work can build on or verify.

Third, the combination of offline-first local persistence, explicit environment classification, and cross-screen data consistency has not been addressed as an integrated engineering concern in a documented academic prototype. Each element exists in isolation in various projects, but their interaction—how a session saved locally under GPS fallback conditions surfaces identically in History, Statistics, and Calendar views—has not been systematically described.

A fourth gap concerns the integration of learning-based prediction models with end-to-end workout tracking systems. While research on deep learning for sensor-based human activity recognition is well established [16, 35], these models are almost exclusively evaluated in controlled laboratory conditions using public benchmark datasets. Their integration into full-stack mobile applications that must handle noisy long-duration workouts, offline persistence, and cross-screen data consistency has not been systematically studied in open, reproducible academic prototypes. Conversely, student and open-source fitness trackers typically rely on hand-crafted heuristics and do not provide a machine learning pipeline built on real user workout data.

Aetron addresses this gap in two stages. In the current pre-thesis prototype, a transparent deterministic rule-based classifier handles environment detection, establishing a reproducible baseline. In the full thesis, anonymised workout sensor logs collected by the application will be used to train and evaluate a supervised activity classifier, providing a direct comparison between the rule-based baseline and a learned model on data collected in real workout conditions.

This project addresses all four gaps by building an academically transparent, integrated prototype in which every architectural and algorithmic decision is documented, reproducible, and evaluated against measurable acceptance criteria.

Chapter 3

System Design and Implementation

3.1 System Overview and Architecture

Aetron is a Flutter mobile application that records outdoor and indoor workout sessions, calculates real-time metrics, stores results locally, and synchronises with a Supabase

cloud backend. Figure 3.1 summarises the end-to-end data flow from session start to cloud synchronisation.

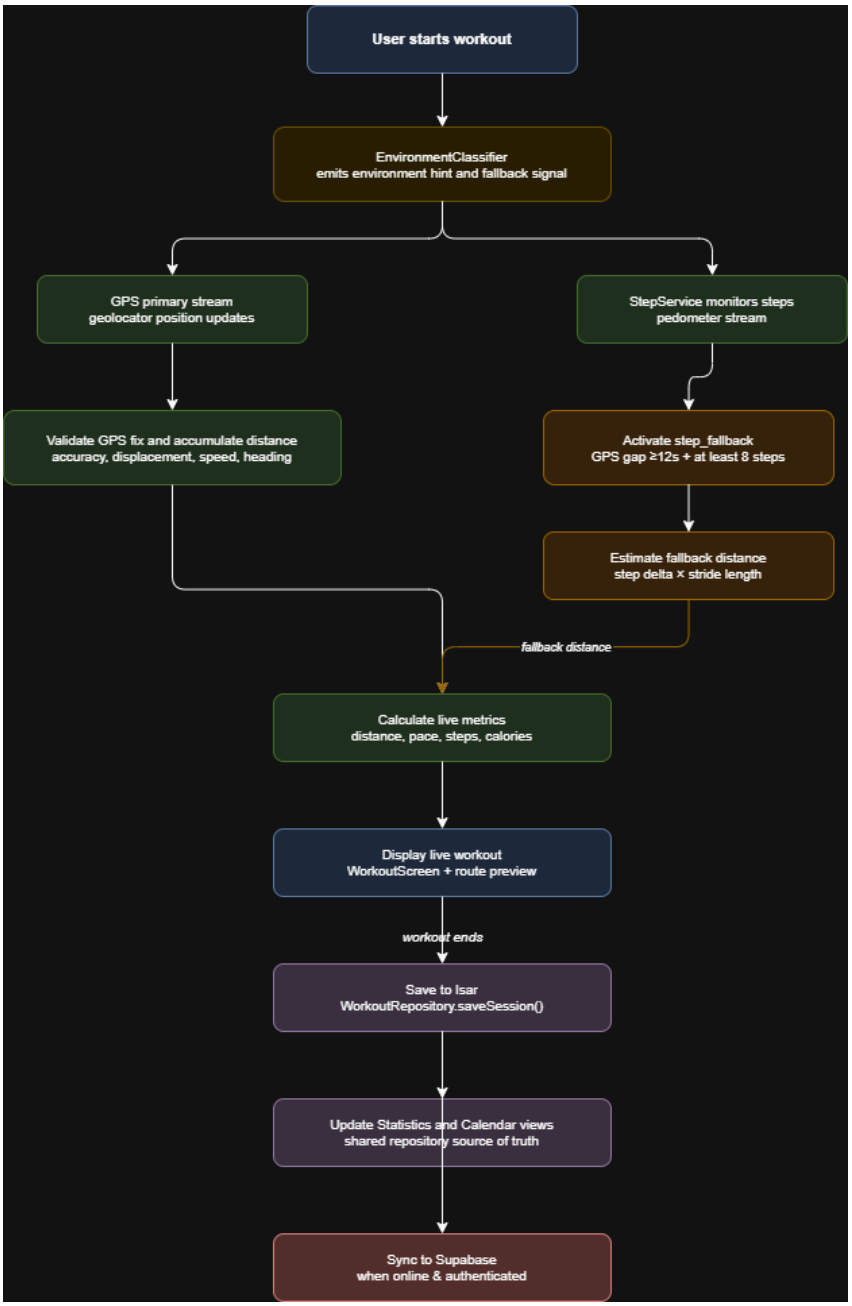


Figure 3.1: System-level end-to-end workout data flow. The `EnvironmentClassifier` provides environment hints, while the runtime fallback is activated by GPS-gap and step-count logic. GPS remains the primary distance source for outdoor workouts; step counting contributes fallback distance when GPS becomes unavailable. For the detailed GPS validation pipeline, see Section 3.4.

To maintain data consistency across sensor reading, local persistence, and multi-screen visualisation, the codebase is organised according to Clean Architecture principles [32]. Figure 3.2 illustrates the separation of the presentation, domain, and data layers, ensuring that the UI depends only on the domain logic and repository interfaces,

rather than directly on the database or sensor implementations.

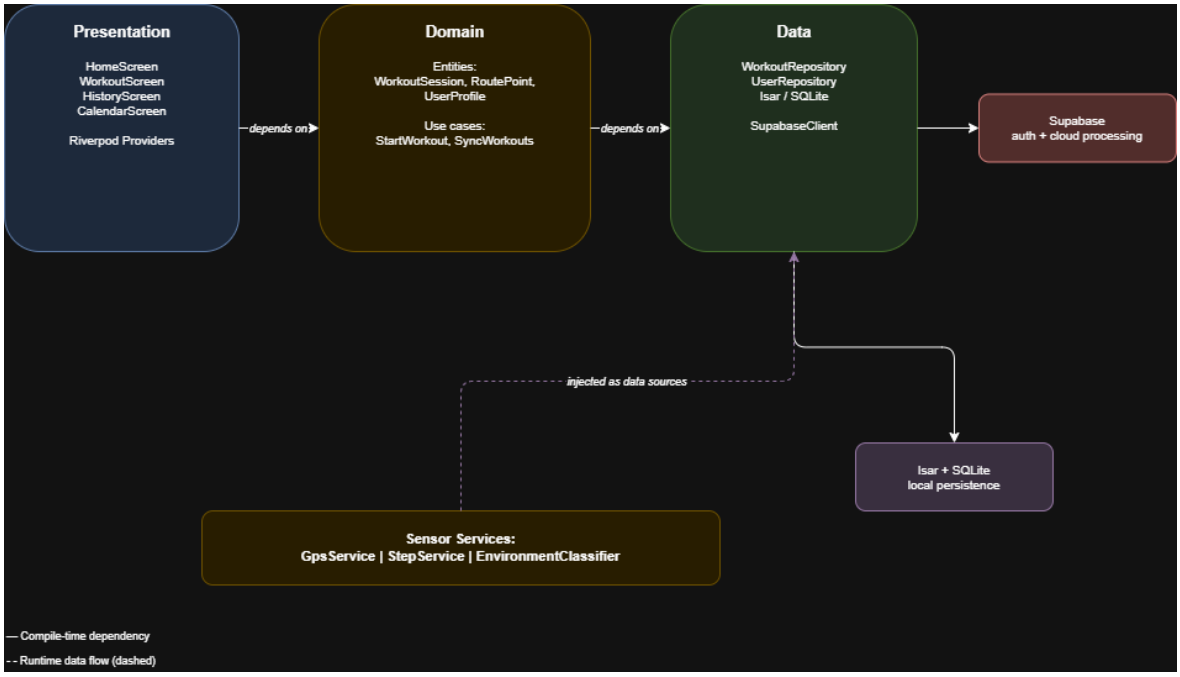


Figure 3.2: System architecture of Aetron following the Clean Architecture pattern [32]. Solid horizontal arrows indicate compile-time dependencies; the domain layer imports no Flutter or platform packages. Solid vertical arrows show runtime data flow. The dashed arrow indicates that sensor services are injected into the data layer as data sources.

3.2 Database Schema

The current implementation uses two local persistence mechanisms. Workout sessions and GPS points are stored in Isar collections because they are read frequently by History, Analytics, Calendar, Summary, and Details screens. User profile data is cached separately in a small SQLite database via `sqflite`. Table 3.1 describes the local workout collection, and Table 3.2 describes the local GPS-point collection.

Table 3.1: Isar `LocalWorkout` collection schema

Column	Type	Description
`id`	Isar Id	Auto-increment local identifier.
`sessionId`	String	Stable UUID shared with Supabase.
`userId`	String	Supabase Auth user identifier.
`activityType`	String	Running, walking, cycling, etc.
`mode`	String	Outdoor or indoor tracking mode.
`startedAt/ended`	DateTime	Session start and stop times.
Duration fields	int	Total and moving duration in seconds.
Metric fields	numeric	Distance, steps, average speed, and calories.
`lapSplitsJson`	String	Encoded lap split data.
`gpsAnalysisJson`	String	Encoded quality analysis.
Route JSON fields	String	Filtered and matched display geometry.
Route status fields	String	Route-correction status and source.
`isSynced`	bool	Whether the local record has been uploaded.

Table 3.2: Isar `LocalGPSPoint` collection schema

Column	Type	Description
`id`	Isar Id	Auto-increment local identifier.
`sessionId`	String	Stable workout UUID.
`workoutId`	int	Link to the local workout row.
`timestamp`	DateTime	Time of the raw/accepted point.
Coordinates	double	Latitude and longitude in WGS-84.
Sensor metadata	double?	Altitude, speed, accuracy, and heading.
`confidence`	String	High, medium, or low confidence label.
`isSynced`	bool	Whether the point has been uploaded.

The SQLite profile cache contains one `user_profile` table with body metrics,

avatar URL, and timestamps. This separation keeps high-volume workout and GPS data in Isar while preserving a lightweight relational cache for profile fields.

3.2.1 Cloud Database Schema (Supabase)

While the local database focuses on immediate read performance and offline capability, the canonical source of truth for the system is the cloud database hosted on Supabase (PostgreSQL). The cloud database is responsible for multi-device synchronisation, data retention, server-side metric processing, and spatial queries via PostGIS. Figure 3.3 illustrates the entity-relationship diagram of the canonical cloud schema.

3.3 Environment Classifier

The environment classifier is a deterministic rule-based component in the core services layer. It is invoked as GNSS position fixes and step deltas arrive. The classifier computes a hint—*outdoor*, *indoor*, or *detecting*—from a rolling window of recent sensor readings and provides fallback signals when GPS quality is weak while steps are increasing. In the current implementation, the rolling window keeps the 10 most recent GPS fixes and requires at least 3 fixes before producing an indoor or outdoor decision; until then, the classifier remains in *detecting* mode. These values were selected empirically during development testing to balance responsiveness against false mode transitions. A window of 10 fixes at 1-second intervals provides approximately 10 seconds of signal history, sufficient to distinguish a genuine environment change from transient GPS fluctuation. The minimum of 3 fixes prevents a classification decision from a single noisy reading while keeping initial detection latency under 5 seconds in practice.

The decision is based on average GPS accuracy, net displacement, average speed, jitter ratio, and whether step count is increasing. Outdoor mode receives a higher score when accuracy, displacement, and speed match the selected activity type. Indoor mode receives a higher score when GPS accuracy is poor, displacement is low, jitter is high, and steps are increasing. For GPS-required activities, this component updates the UI-facing `environmentHint`; the actual step fallback is activated by the tracking engine when GPS has no accepted fix for about 12 seconds and at least 8 steps are observed.

Table 3.3 summarises the specific thresholds used by the environment classifier and their supporting literature.

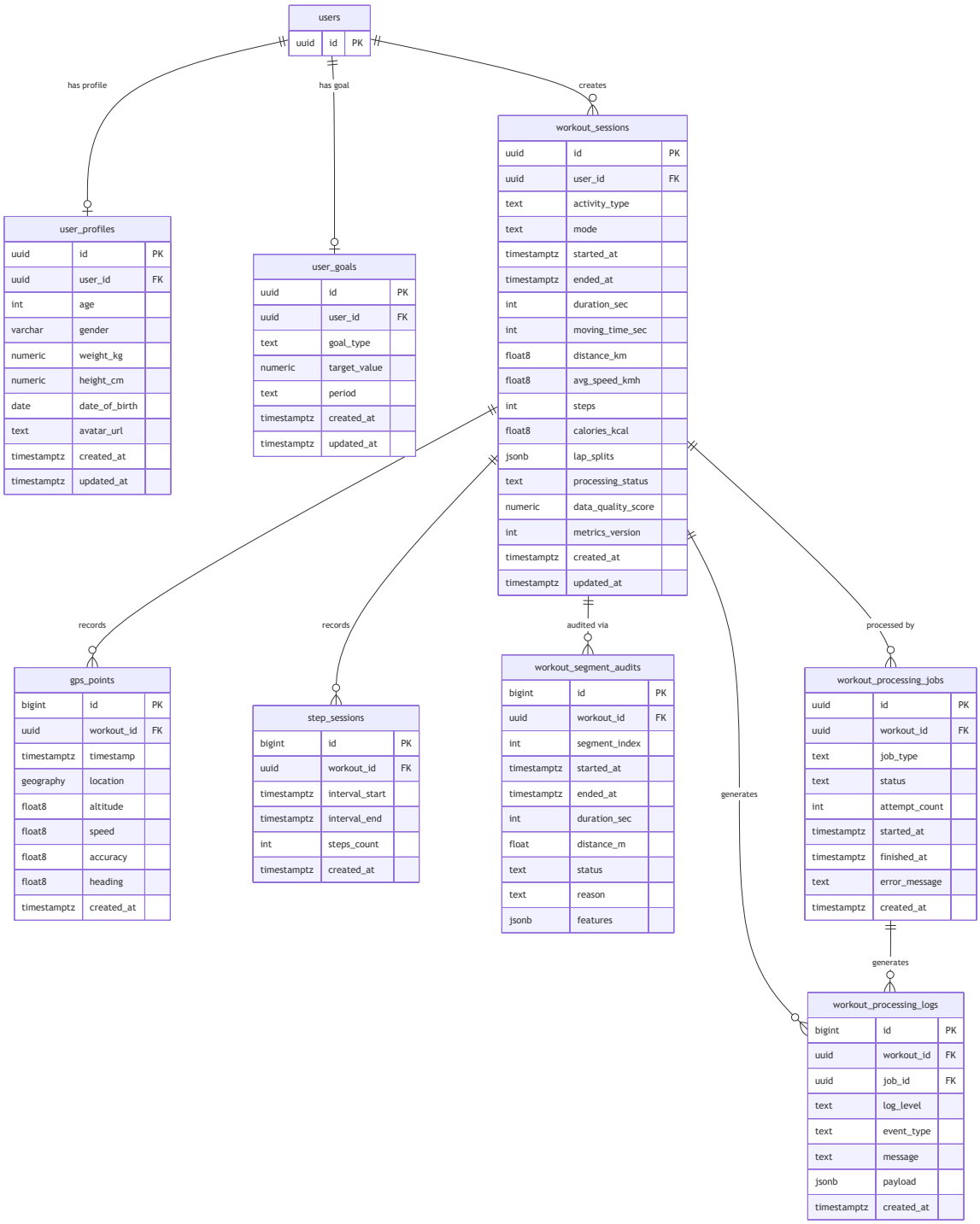


Figure 3.3: Entity-Relationship Diagram (ERD) of the Aetron cloud database schema. The schema separates core workout records (`workout_sessions`) from high-volume tracking telemetry (`gps_points`, `step_sessions`) and backend processing pipelines (`workout_processing_jobs`).

Table 3.3: Threshold parameters for the environment classifier and their supporting literature.

Metric	Value	Literature Support
GPS accuracy	$\leq$ 20 m (outdoor) $\geq$ 35 m (indoor)	Informed by Merry & Bettinger [7] and Zandbergen & Barbeau [12]
Speed	$\geq$ 1.5 m/s (outdoor) $\leq$ 0.4 m/s (indoor)	Directly supported by clinical gait norms [36, 37, 38]
Displacement	$\geq$ 8 m (outdoor) $\leq$ 3 m (indoor)	Informed by GNSS error ranges [12, 39]
Jitter ratio	$\leq$ 0.3 (outdoor) $\geq$ 0.6 (indoor)	Conceptually informed by GNSS variance features [40]

GPS accuracy thresholds were informed by Merry and Bettinger [7] and Zandbergen and Barbeau [12], which report median outdoor errors of 7–13 m and urban-canyon or indoor errors of 30–50 m and beyond. The thresholds of $\leq$ 20 m (outdoor) and $\geq$ 35 m (indoor) were selected to sit within these empirically reported ranges and were validated during development testing.

The speed threshold of $\leq$ 0.4 m/s for the indoor classifier signal corresponds to gait velocities classified as household ambulation in clinical biomechanics literature [36, 37, 38], representing a distinctly different locomotion behaviour from typical outdoor walking and therefore a reliable indicator of minimal real-world displacement. The outdoor speed threshold of $\geq$ 1.5 m/s is higher than comfortable walking speed, serving to confirm actual displacement rather than stationary GPS drift.

The displacement thresholds of $\geq$ 8 m (outdoor) and $\leq$ 3 m (indoor) over the rolling window were set empirically during development testing. These values are consistent with the GPS horizontal error ranges reported by Zandbergen and Barbeau [12] (median 5–8.5 m outdoors) and with practice in GPS jitter filtering, where displacements below the sensor's nominal error floor are considered indistinguishable from stationary noise [7, 39]. No single published study provides identical threshold values; these are tuned empirical parameters calibrated on the target device and test environment.

The jitter ratio, defined as positional variance relative to net displacement over the rolling window, captures the signal instability characteristic of indoor GNSS multipath described by Zhu et al. [40]. The threshold values ($\leq$ 0.3 outdoor, $\geq$ 0.6 indoor) are empirically derived parameters tuned during development testing; no published study

provides equivalent threshold values for this specific formulation.

Figure 3.4 illustrates the decision logic.

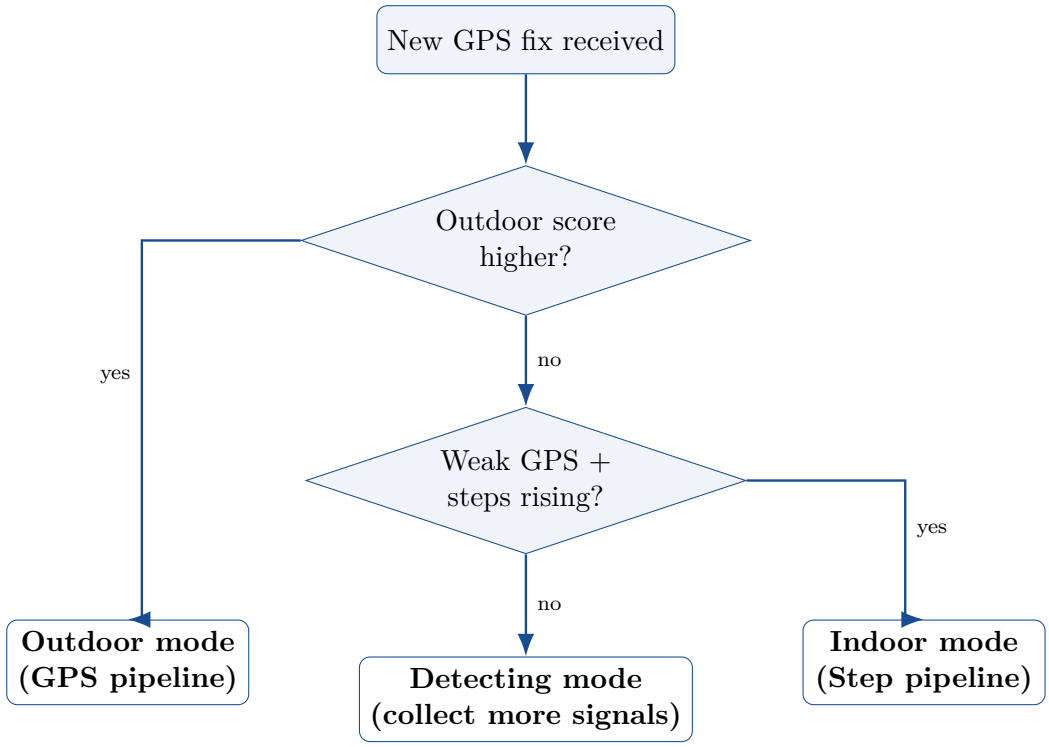


Figure 3.4: Environment classifier decision logic (simplified). The full scoring function uses five signals over a rolling window of ten GPS fixes; see Section 3.3 for details.

This deterministic approach was chosen over a machine-learning activity classifier because it requires no training data, its decisions are fully interpretable, and it adds no inference latency [14]. The codebase also contains a `kDebugLocationMode` flag controlled by `AETRON_DEBUG_LOCATION`; it is disabled by default and relaxes thresholds only when explicitly enabled for mock-location development. In debug mode (`AETRON_DEBUG_LOCATION=true`), the initial-fix accuracy threshold is relaxed to 50 m for all activity types, the minimum displacement threshold is reduced to 0.1 m, and the speed sanity limit is raised to 150 m/s. These relaxed parameters allow mock-location testing with simulated GPS coordinates without triggering the validation pipeline's rejection logic. The flag is disabled by default in all release builds. All evaluation results reported in Chapter 4 use the normal production thresholds.

Figure 3.1 shows the end-to-end data flow. Section 3.4 describes the GPS validation pipeline in detail.

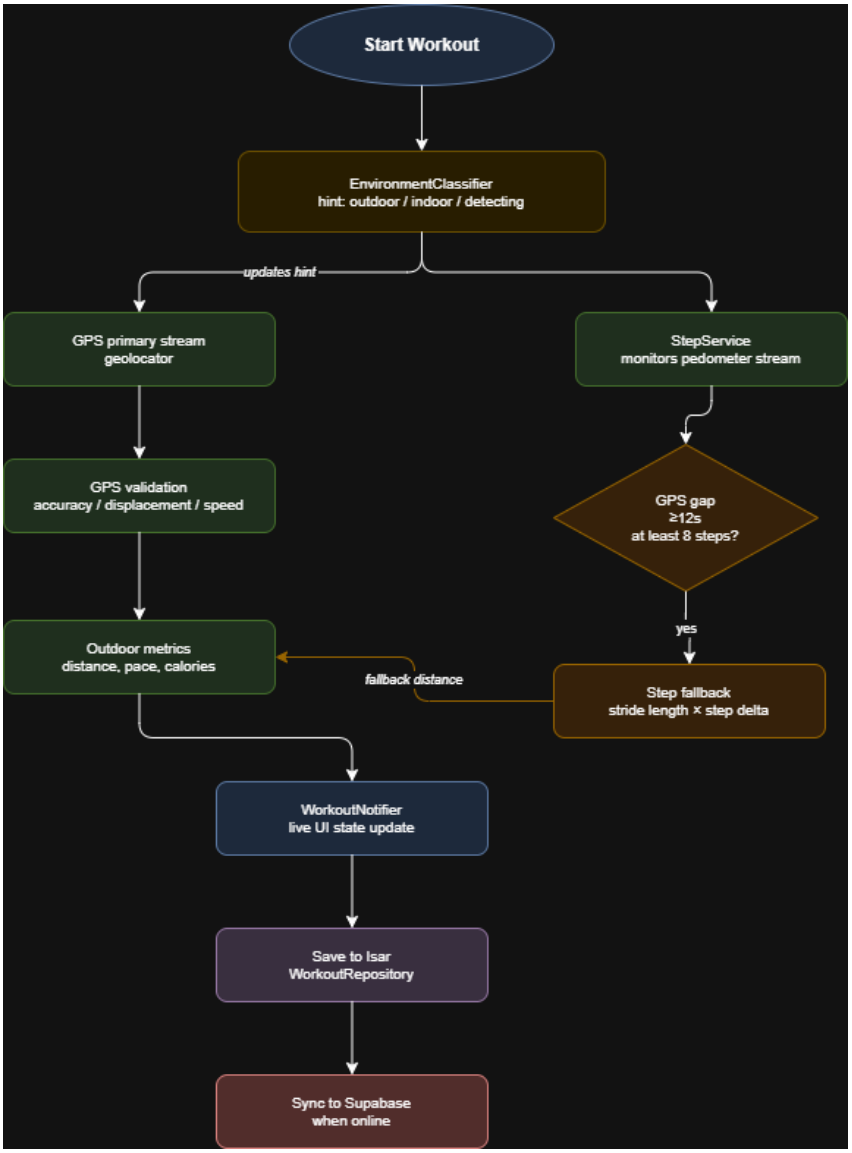


Figure 3.5: Tracking-engine detail—GPS-primary pipeline with step-count fallback. This figure expands the signal-acquisition and tracking-environment stages shown in Figure 3.1. The `EnvironmentClassifier` updates the environment hint, while `step_fallback` is activated only when the GPS stream has no accepted fix for about 12 seconds and at least 8 steps are observed.

3.4 Outdoor GPS Tracking Pipeline

When outdoor mode is active, the location service emits a stream of `Position` objects from `geolocator`. Each incoming position fix must pass four validation stages before the pipeline adds it to the distance accumulator:

1. **Initial-fix gate:** the first route point is accepted only after the reported accuracy is good enough for the selected activity.
2. **Minimum displacement:** very small movements are treated as GPS jitter and do not update the route distance.
3. **Accuracy gate:** low-quality fixes are soft-rejected and used only for preview when appropriate.
4. **Speed and heading checks:** hard GPS spikes are rejected using sensor speed, implied speed, and abrupt bearing-change checks.

Table 3.4 lists the concrete validation parameters used by the current implementation.

Table 3.4: GPS validation parameters used in the outdoor tracking pipeline

Parameter	Value	Purpose
Initial fix accuracy	Walking: 18 m; Running: 16 m; Cycling: 20 m	Prevents the route from starting with a weak first GPS point.
Route accuracy gate	Walking: 50 m; Running: 42 m; Cycling: 32 m	Soft-rejects low-quality fixes before distance accumulation.
Minimum segment	Walking: 0.35 m; Running: 0.30 m; Cycling: 1.5 m	Filters tiny movements that are likely GPS jitter.
Hard GPS spike speed	80 m/s	Hard physical-impossibility gate, not an activity-specific speed limit.
Stale signal gap	>5 s	Preserves the route point after a gap but does not add distance for the gap.
Heading spike	Bearing change >120° and segment <20 m	Removes short zig-zag spikes from the route.

Three sources informed the threshold values in Table 3.4. First, published GPS

accuracy benchmarks for consumer smartphones in urban environments informed the order of magnitude: Merry and Bettinger [7] report median horizontal errors of 3–10 m in open conditions and up to 30–50 m in urban canyons, which sets the upper bound for the route accuracy gate (50 m for walking, 42 m for running, 32 m for cycling). Tighter thresholds for cycling reflect the expectation that cyclists typically operate in less occluded environments.

The per-activity initial-fix thresholds (18 m for walking, 16 m for running) were set empirically across 15 development sessions on a Samsung Galaxy A52 (Android 13) in Ho Chi Minh City. The slightly lower threshold for running reflects the observation that running sessions in the test environment tended to start in open areas (parks, roads) where GPS lock quality was consistently good, whereas walking sessions included more mixed environments. These values are device- and environment-specific tuned defaults; they may require adjustment for different hardware or geographic conditions.

All hard physical-impossibility limits (80 m/s speed gate, 120° bearing spike) are derived from the physical constraints of human-powered locomotion and are not activity-specific.

The pipeline appends each accepted fix to the session route list and adds the segment distance to the live workout state. The same list is passed to Flutter Map as a `Polyline` layer for live route rendering.

3.4.1 Distance Calculation

Distance between consecutive accepted fixes is computed using the Haversine formula (Equation 2.3). In the implementation, this is delegated to the `geolocator` package's built-in `Geolocator.distanceBetween` function, which applies the WGS-84 ellipsoidal model internally. Listing 3.1 shows how the outdoor tracking pipeline computes both the raw distance used for accumulation and the route segment used for display.

Listing 3.1: GPS distance accumulation in `WorkoutTrackingEngine.evaluateGpsUpdate` (`workout_tracking_engine.dart`)

```
// rawDistanceAnchor = last accepted point or distance
   anchor after resume
final rawSegmentMeters = Geolocator.distanceBetween(
  rawDistanceAnchor.latitude,
  rawDistanceAnchor.longitude,
  livePoint.latitude,
  livePoint.longitude,
);

```

```
// routeSegmentMeters used for route display and smoothing
    only
final routeSegmentMeters = Geolocator.distanceBetween(
  lastPoint.latitude,
  lastPoint.longitude,
  livePoint.latitude,
  livePoint.longitude,
);

// Only rawSegmentMeters is added to the distance accumulator
final segmentMeters = rawSegmentMeters;
```

3.4.2 Pace and Speed Calculation

Average speed and pace are derived from the accumulated distance and the elapsed moving time, excluding pause intervals, as per Equations 2.8.

3.4.3 Route Smoothing and Display Cleanup

The validated distance source is kept separate from the displayed route. For visual stability, `GpsSmoothingService` classifies each accepted point as high, medium, or low confidence and blends it with the previous smoothed point. Stationary updates can be frozen to prevent the map marker from drifting while the user is stopped. Before a saved route is shown in the summary or details screen, `route_display_sanitizer.dart` removes tiny segments, sharp spikes, and excessive zig-zags while preserving meaningful corners. This improves map readability without hiding the fact that the underlying sensor data remains noisy.

3.5 Indoor Step Tracking Pipeline

When indoor mode is active, `StepService` subscribes to the `pedometer` plugin's step-count stream [3]. The pedometer reports a monotonically increasing cumulative step count since the last device reboot [18]. At session start, the baseline count is recorded. At each update, the session step count is:

$$N_{\text{steps}} = C_{\text{current}} - C_{\text{baseline}} \tag{3.1}$$

Indoor distance is estimated by Equation 2.6 using the stride-length coefficients

from Equation 2.7, with the user's height loaded from the profile. Listing 3.2 shows the `computeStrideLength` function, which selects gender-specific height-based coefficients (Equation 2.7) and falls back to fixed defaults when height is unavailable.

Listing 3.2: Stride-length computation in `record_providers.dart`

```dart
// Returns a fixed default when height is unavailable
double _defaultStrideLength(String activityType, String? gender) {
  final isRunning = activityType.toLowerCase().contains('run');
  final normalizedGender = _normalizeGender(gender);
  if (isRunning) {
    return normalizedGender == 'female' ? 0.85 : 0.93;
  }
  return normalizedGender == 'female' ? 0.68 : 0.73;
}

// Computes stride length from height using Equation (2.7) coefficients
double computeStrideLength({
  required String activityType,
  double? heightCm,
  String? gender,
}) {
  if (heightCm == null || heightCm <= 0) {
    return _defaultStrideLength(activityType, gender);
  }
  final isRunning = activityType.toLowerCase().contains('run');
  final normalizedGender = _normalizeGender(gender);
  // Running: 0.67H (male), 0.62H (female)
  // Walking: 0.43H (male), 0.40H (female)
  final factor = isRunning
      ? (normalizedGender == 'female' ? 0.62 : 0.67)
      : (normalizedGender == 'female' ? 0.40 : 0.43);
  final raw = heightCm * factor / 100;  // convert cm to m
  return raw.clamp(0.50, 1.50);         // enforce physiological bounds
}
```

3.6 Calorie Estimation

The calorie estimation formula implemented in Aetron is a simplified distance-based model:

$$E = W \times D_{\text{km}} \times k \times g \tag{3.2}$$

where W is body mass in kilograms, D_{km} is distance in kilometres, k is an activity coefficient (1.05 for running, 0.92 for walking and cycling), and g is a small gender adjustment factor (0.95 for female profiles).

The activity coefficient $k = 1.05$ for running and $k = 0.92$ for walking were set as prototype defaults to bring Aetron's output into the same order of magnitude as commercial applications. These are heuristic constants, not values derived from indirect calorimetry or a representative user sample. The gender adjustment factor $g = 0.95$ applies a small downward correction for female profiles, following the convention that women generally have lower absolute energy expenditure at the same body mass and pace.

Because these coefficients were chosen by inspection rather than calibrated against physiological data, calorie output should be interpreted as a rough indicator of energy expenditure, not a physiologically grounded measurement. The net active energy literature value of 0.63–0.75
,kcal/kg/km [22] represents active expenditure only; the prototype coefficient exceeds this range because it accounts for a resting metabolic contribution. Given that even dedicated wearables with heart rate sensors and proprietary algorithms exhibit systematic calorie estimation errors of approximately 28% for Apple Watch [28] and exceeding 30% across consumer wearable brands generally [29], and that smartphone-only tracking cannot measure heart rate, oxygen consumption, or individual metabolic efficiency, this deterministic approach prioritises cross-screen consistency and algorithmic transparency over physiological precision. A more rigorous derivation—involving treadmill-based indirect calorimetry with representative participants—is left for future work.

All calorie values are labelled "estimates" in the UI and are not presented as medical-grade measurements. The primary design goal is that the same workout record yields identical calorie values when viewed in History, Statistics, Calendar, and Detail screens—a data consistency requirement that deterministic calculation satisfies by construction.

3.7 Synchronisation Strategy

The synchronisation design follows a local-first approach:

1. On session end, the `WorkoutRepository` writes the completed `WorkoutSession` to Isar with `isSynced = false`.
2. The repository attempts to upsert the session into Supabase `workout_sessions` when the user is authenticated and the network is reachable.
3. Raw GPS points and step intervals are uploaded in batches to `raw_gps_points` and `raw_step_intervals`.
4. Processing jobs are inserted into `workout_processing_jobs`; logs are written to `workout_processing_logs`.
5. On login from a new device, remote sessions are fetched and merged into the local Isar store by UUID.

This design guarantees that no completed workout is lost due to network failure or authentication expiry.

3.8 Backend Processing Pipeline

As a proof-of-concept for server-side processing, the Supabase backend includes a set of Edge Functions beyond the core authentication and synchronisation endpoints. The `workouts-start` function creates a session shell when recording begins; `gps-track` accepts batches of raw GPS points during a session; and `workouts-end` finalises the client-submitted summary. Two queued worker functions extend this flow at the prototype level: `deterministic-finalize-worker` recomputes canonical metrics from stored raw data, and `route-correction-worker` records route confidence metadata for future map-matching experiments.

This backend pipeline is a proof-of-concept and is not part of the primary acceptance criteria for this pre-thesis stage. Its purpose is to establish an auditable upload trail and demonstrate that server-side reprocessing is architecturally feasible without blocking the mobile client. Production-grade route map-matching, anomaly detection, and conflict resolution are deferred to the full thesis stage.

3.9 Cross-Screen Data Consistency

All consumer modules—History, Analytics, Calendar, Summary, and Details—query workout data through the same `WorkoutRepository`. None maintains its own independent copy of workout records; instead:

- **History:** reads local sessions ordered by `startedAt` descending, with optional date-range filtering.
- **Analytics:** computes aggregates such as total distance, calories, session count, weekly trend, records, and activity distribution from the same session list.
- **Calendar:** groups sessions by local date and renders active-day markers.
- **Summary and Details:** reuse the saved session, lap splits, route geometry, and validity metadata rather than recalculating separate values.

Because all modules share the same repository source of truth, a workout saved after a session immediately appears consistently across the app on the next screen load.

3.10 Workout Recording UX Flow

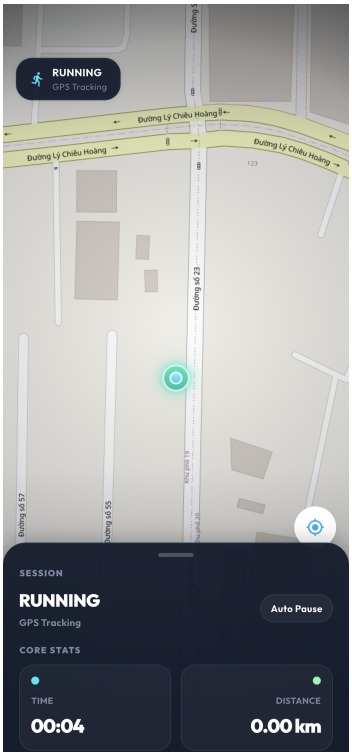
The user experience is designed around a seamless transition from the Home dashboard to active recording and post-workout analysis. The Home screen serves as a high-level hub, visualising core load and calorie burn via an "Energy Flux" indicator alongside a 7-day analytics trend. From Home, navigating to the Activity screen allows users to select their workout type. Prior to recording, a preparation screen clearly displays the environment mode (outdoor/indoor) determined by the classifier and confirms GPS signal readiness.

During a session, the Workout Recording module presents a live route polyline, a workout timer, and real-time metrics (distance, pace, calories). Upon session completion, the data is saved locally and immediately surfaces across the History, Analytics, and Calendar modules via the shared repository source of truth. Figure 3.6 illustrates the key screens of this recording flow.

The Profile and Settings modules support this flow by managing user anthropometric data (used for stride length and calorie estimation), fitness goals, and application preferences.



(a) Home dashboard



(b) Workout recording

Figure 3.6: Key screens of the workout UX flow: the Home dashboard (left) displaying goal progress, and the Workout recording screen (right) showing live metrics and map rendering.

Chapter 4

Experiment Results and Evaluation

4.1 Comparison with Reference Applications

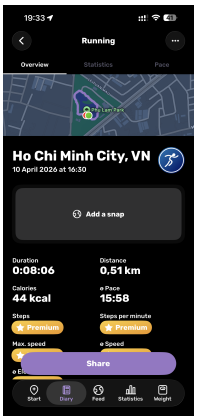
Objective: Contextualise Aetron's measurements against established commercial peers, rather than determining absolute accuracy against a surveyed ground truth.

Setup: Each application ran on a dedicated smartphone to avoid interference. All applications were started and stopped concurrently using the same gestures. Aetron used its standard GPS validation thresholds.

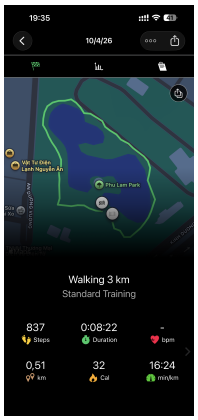
Results and Remarks: Aetron recorded 0.52 km, sitting securely within the 0.51–0.56 km range reported by the commercial apps. This confirms that Aetron's GPS filtering pipeline produces distance estimates comparable to proprietary industry algorithms under identical conditions. Calorie and duration discrepancies reflect proprietary model differences and minor manual synchronisation delays across devices, which are expected constraints of this comparative methodology.

Table 4.1: Recorded metrics across six fitness applications on the same walking route (≈510 m, outdoor urban, Route B). Strava calorie is marked N/A because the captured summary screenshot did not display calorie expenditure.

Application	Duration	Distance (km)	Calories (kcal)	Avg pace (min/km)	Status
FITAPP	08:06	0.51	44	15:58	Recorded
Goals-Fitness	08:22	0.51	32	16:24	Recorded
Running (Adidas)	07:57	0.56	32	14:08	Recorded
Nike Run Club	08:09	0.51	28	15:47	Recorded
Strava	08:16	0.52	N/A	15:54	Recorded
Aetron (this work)	08:21	0.52	34	16:03	Recorded



(a) FITAPP



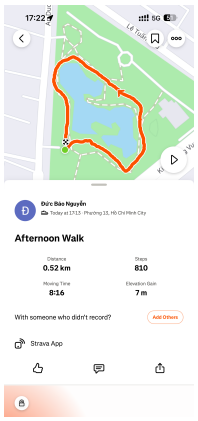
(b) Goals-Fitness



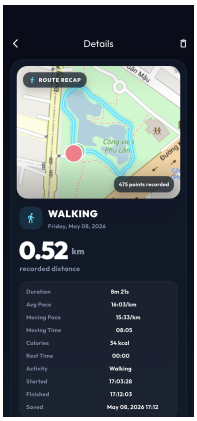
(c) Running by Adidas



(d) Nike Run Club



(e) Strava



(f) Aetron

Figure 4.1: Route visualisation and recorded summary comparison across reference applications and Aetron on the same approximately 510 m walking route.

4.1.1 GPS Distance Benchmark using MAPE

While the comparative experiment in Section 4.1 situates Aetron within the range of existing fitness applications, it does not provide an absolute accuracy measure because no surveyed ground-truth distance was available. To establish a reproducible benchmark, a separate evaluation based on a pre-measured reference route is defined.

In this benchmark, Aetron is tested on a standard reference distance where the true distance is known. For each trial, the application records the total GPS distance, and the mean absolute percentage error (MAPE) is computed as

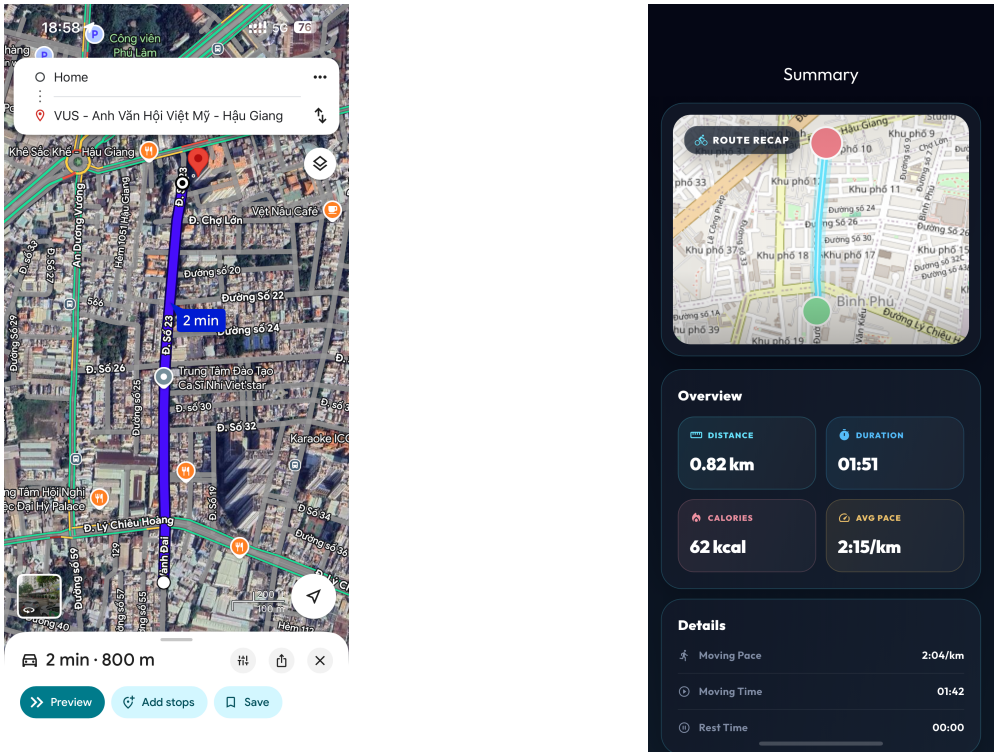
$$\text{MAPE} = \frac{1}{N} \sum_{i=1}^{N} \left| \frac{d_i - d_{\text{ref}}}{d_{\text{ref}}} \right| \times 100\%,$$

where d_i is the recorded distance for trial i and d_{ref} is the known reference distance. Following the methodology of Tierney et al. [41], this metric provides a single summary value for GPS distance accuracy across multiple repetitions.

MAPE values are interpreted according to the classification proposed by Scott et al. [42], where values below 5% are considered good, 5–10% moderate, and above 10% poor. This interpretation is consistent with the benchmark reported by Gilgen-Ammann et al. [43], who found MAPEs of 3.2%–6.1% across eight GNSS-enabled sport watches under similar outdoor conditions.

For this pre-thesis submission, an initial MAPE benchmark was executed using Google Maps as a reference standard for a route distance of 800 m ($d_{\text{ref}} = 0.80$ km). As shown in Figure 4.2, Aetron recorded a distance of 0.82 km ($d_1 = 0.82$ km). For $N = 1$ trial, this results in an absolute discrepancy of 0.02 km, equating to a MAPE of 2.5%.

This low error margin falls well within the "good" category (below 5%), demonstrating that Aetron's GPS filtering pipeline maintains high absolute accuracy and resists significant cumulative drift during tracking sessions. Further execution of this benchmark across multiple repetitions is reserved for the full thesis stage.



(a) Google Maps Reference (b) Aetron Recording

Figure 4.2: Initial MAPE benchmark validation comparing Google Maps distance and Aetron's recorded distance.

4.2 Developer Walkthrough Review

Objective: Qualitatively assess the application's interface against baseline usability heuristic dimensions.

Setup: The author conducted a structured developer walkthrough of the five main application modules. A formal, externally-validated usability study using the System Usability Scale (SUS) is planned for the full thesis stage.

Results and Remarks: The interface demonstrates strong *visual consistency* and *navigation clarity* through a uniform dark theme and a persistent bottom navigation bar. *Workout readability* is excellent with high-contrast active recording metrics. However, *map usability* during indoor sessions and *action clarity* regarding permission requests require further refinement.

4.3 Software Verification

Objective: Verify the functional correctness of the most critical, failure-prone logic components at the unit level.

Setup: A focused suite of unit and service tests was defined and executed via the `flutter test` framework. Negative-path coverage and emulator-based integration tests are deferred to the full thesis stage.

Results and Remarks: All defined test cases passed successfully, confirming the unit-level stability of the prototype's core tracking logic, validation gates, and data processing routines.

4.4 Environment Classification Field Observation

Objective: Verify the environment classifier's transition behaviour (O1 — Adaptive Tracking Pipeline) in a real-world context.

Setup: The tester started the application outdoors and walked into an interior corridor while monitoring the environment hint via the debug overlay. This was repeated three times.

Results and Remarks: The transition time from outdoor to indoor mode averaged ≈ 14 seconds. This latency accurately reflects the theoretical delay imposed by the classifier's rolling window algorithm (Section 3.3).

4.5 Indoor Step Counting Evaluation

Objective: Evaluate the baseline accuracy of the OS-level step-counter used for indoor tracking (O1 — Adaptive Tracking Pipeline).

Setup: The tester walked along a straight indoor corridor at a steady pace. Each trial consisted of 100 manually counted steps, repeated five times.

Results and Remarks: The mean absolute error was 3.0% (Table 4.2), which successfully meets the 5% acceptance threshold. Trial 1 recorded a higher error of 7.0%, likely due to an initial counting offset at session start; the remaining four trials fell within 0–4%, and the overall mean of 3.0% satisfies the acceptance threshold. Broader validation across multiple participants and devices is deferred to the full thesis stage.

Table 4.2: Indoor step-counting accuracy over five trials of 100 manually counted steps

Trial	**App-reported steps**	**Error (%)**
1	93	7.0%
2	103	3.0%
3	99	1.0%
4	100	0.0%
5	96	4.0%
Mean	**98.2**	**3.0%**
Std Dev	**3.8**	**2.7%**

4.6 Evaluation Summary

This section summarises the current evaluation status of the four project objectives (O1–O4) at the pre-thesis stage. Table 4.3 consolidates the acceptance criteria and results against the objectives defined in Chapter 1.

Table 4.3: Evaluation results against acceptance criteria

Obj.	Criterion	Result	Pass	Note
O1	Environment transition; GPS distance within range; step error $< 5\%$	Transition $\approx$14 s; 0.52 km vs. 0.51–0.56 km; MAPE 2.5%; step error 3.0%	Pass	Verified via field observation, comparative benchmark, and manual step trials (Sections 4.1–4.5).
O2	Metric update ≤ 1 s	Confirmed ≤ 1 s	Pass	UI state refresh frequency verified during live sessions.
O3	Persist after force-close; sync after re-login	Data recovered (10/10); sync successful	Pass (core); Partial (backend)	Local Isar commit verified before network sync. Backend post-processing workers deployed as proof-of-concept; production-grade validation deferred to full thesis (Section 3.8).
O4	Cross-screen consistency; modular architecture	Metrics consistent; architecture reviewed	Pass	Consistent metric views verified across all UI modules; presentation, domain, data, and service layers confirmed separated.

With the successful execution of these evaluations, the pre-thesis prototype has satisfied all four core objectives. O1 is validated through comparative GPS benchmarks, a MAPE benchmark against a known reference route, and manual indoor step-counting trials. O2 is confirmed by observed UI refresh latency during live sessions. O3 is fully satisfied for the local persistence and cloud upload criteria; backend post-processing remains at proof-of-concept level and is deferred to the full thesis. O4 is confirmed through cross-screen metric verification and an architectural review.

The primary expected source of GPS variability is multipath reflection in dense

urban-canyon environments, consistent with prior literature [7, 12]. The validation pipeline aims to reduce but does not eliminate this effect.

Chapter 5

Conclusion and Future Work

5.1 Conclusion

This project shows that a smartphone-only fitness tracker can be built as an academically transparent prototype without relying on proprietary tracking algorithms. The main value of Aetron is not a novel positioning formula, but the integration of noisy mobile sensors, deterministic validation, offline-first storage, cloud synchronisation, and consistent multi-screen presentation into one inspectable workflow.

The prototype demonstrates several lessons for the full thesis stage. First, deterministic validation gates make GPS behaviour easier to explain and debug than raw point accumulation. Second, indoor tracking can provide a useful fallback through OS step counting, but its distance accuracy remains bounded by stride-length approximation. Third, local-first persistence is essential for workout applications because sensor capture and network availability cannot be assumed to succeed at the same time.

The primary limitation of this prototype is that GPS accuracy in dense urban environments can inflate recorded distance and distort route shape, consistent with prior literature [12]. Indoor distance estimation accuracy is bounded by the error in the stride-length approximation for uncalibrated users. These limitations are inherent to smartphone-only tracking without external sensors and are acknowledged in the application's documentation.

5.2 Limitations

GPS accuracy in urban environments. The validation pipeline reduces but does not eliminate positional errors caused by multipath reflections in dense building environments. Route shape can still deviate from the real path near tall structures, causing small but measurable distance errors.

Stride-length approximation. The gender-specific stride coefficients (Equation 2.7) represent population averages. Individual users whose stride length deviates significantly from the population mean will experience proportionally larger indoor distance errors. Calibration against a known-distance reference would reduce this error.

Calorie estimation uncertainty. The distance-based calorie formula is a population-average approximation. Actual energy expenditure depends on individual physiology (running economy, heart rate, metabolic rate) that a smartphone cannot measure. Calorie values are approximate and should not be used for medical decisions.

Background tracking reliability. OS battery-optimisation policies can suspend location or sensor services if the application is moved to the background. The current implementation uses a wakelock to mitigate this, but complete reliability across all Android OEM variants and battery modes cannot be guaranteed without further device-specific testing.

5.3 Future Work

ML-based GPS Anomaly Detection. The current filtering pipeline relies on rigid, deterministic thresholds (e.g., maximum distance jumps or accuracy bounds) to discard invalid GPS fixes. While effective in general conditions, this rule-based approach can struggle with complex multipath reflections. A major proposed direction for the full thesis is to augment or replace the deterministic classifier with an unsupervised, lightweight machine learning model, such as an Isolation Forest or One-Class SVM. This model would evaluate a four-dimensional input vector for each fix—comprising accuracy, speed, displacement, and jitter—to output a continuous confidence score indicating the likelihood of an anomaly. Executed entirely on-device via TensorFlow Lite within Flutter, this mechanism requires no labelled training data and preserves user privacy. Crucially, this addresses the learning-based versus rule-based dichotomy highlighted in Section 2.5.2. It positions the full thesis to make a distinct contribution by presenting a rigorous, comparative evaluation between the current deterministic baseline and the proposed ML-driven anomaly detection approach.

Stronger route filtering model. The current route cleanup uses deterministic confidence-based smoothing and bearing-aware spike removal. A controlled comparison with a full Kalman filter [15] or map-matching baseline is intentionally left for the full thesis stage, where more routes, devices, and ground-truth references can be collected.

User stride calibration. Adding a one-time calibration walk over a known indoor distance would allow the application to compute a personalised stride coefficient, replacing the population-average default and reducing indoor distance error for individual

users.

Activity-adaptive calorie model. The current distance-based calorie model is deterministic but simple. Integrating GPS-derived speed, cadence from the pedometer, activity type, and user profile history would produce more consistent calorie estimates while keeping the system smartphone-only.

Wearable Integration for Physiological Metrics. The current smartphone-only design fundamentally limits calorie estimation accuracy, as it lacks access to physiological signals like heart rate. A concrete area for future work is integrating with smartwatch APIs (e.g., Health Connect or Wear OS) to ingest real-time heart rate data. This would allow the transition from the current deterministic distance-based heuristic to a more robust, physiologically-grounded energy expenditure model.

Production route correction. The current backend includes a route-correction worker and confidence metadata, but broader validation is still needed before treating matched geometry as fully production-grade across dense urban, park, and indoor-edge cases.

Bibliography

[1] Google LLC. (2023) Flutter: Build apps for any screen. [Online]. Available: https://flutter.dev/

[2] Baseflow. (2024) geolocator – flutter package. [Online]. Available: https://pub.dev/packages/geolocator

[3] Julian Oes. (2024) pedometer – flutter package. [Online]. Available: https://pub.dev/packages/pedometer

[4] Remi Rousselet. (2024) Riverpod: A reactive caching and data-binding framework. [Online]. Available: https://riverpod.dev/

[5] Flutter Community. (2024) sqflite – flutter package. [Online]. Available: https://pub.dev/packages/sqflite

[6] Supabase Inc. (2023) Supabase documentation. [Online]. Available: https://supabase.com/docs

[7] K. Merry and P. Bettinger, "Smartphone GPS accuracy study in an urban environment," *PLOS ONE*, vol. 14, no. 7, p. e0219890, 2019.

[8] A. Brajdic and R. Harle, "Walk detection and step counting on unconstrained smartphones," *Proceedings of the 2013 ACM International Joint Conference on Pervasive and Ubiquitous Computing*, pp. 225–234, 2013.

[9] B. E. Ainsworth, W. L. Haskell, S. D. Herrmann, N. Meckes, D. R. Bassett, C. Tudor-Locke, J. L. Greer, J. Vezina, M. C. Whitt-Glover, and A. S. Leon, "2011 compendium of physical activities: A second update of codes and MET values," *Medicine & Science in Sports & Exercise*, vol. 43, no. 8, pp. 1575–1581, 2011.

[10] World Health Organization. (2024) Physical activity. [Online]. Available: https://www.who.int/news-room/fact-sheets/detail/physical-activity

[11] J. Yun, C. Lim, and B. Park, "Inherent limitations of smartphone GNSS positioning and effective methods to increase the accuracy utilizing dual-frequency measurements," *Sensors*, vol. 22, no. 24, p. 9879, Dec. 2022.

[12] P. A. Zandbergen and S. J. Barbeau, "Positional accuracy of assisted GPS data from high-sensitivity GPS-enabled mobile phones," *Journal of Navigation*, vol. 64, no. 3, pp. 381–399, 2011.

[13] R. W. Sinnott, "Virtues of the haversine," *Sky and Telescope*, vol. 68, no. 2, pp. 158–159, 1984.

[14] R. Harle, "A survey of indoor inertial positioning systems for pedestrians," *IEEE Communications Surveys & Tutorials*, vol. 15, no. 3, pp. 1281–1293, 2013.

[15] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.

[16] J. Wang, Y. Chen, S. Hao, X. Peng, and L. Hu, "Deep learning for sensor-based activity recognition: A survey," *Pattern Recognition Letters*, vol. 119, pp. 3–11, Mar. 2019.

[17] F. J. Ordóñez and D. Roggen, "Deep convolutional and LSTM recurrent neural networks for multimodal wearable activity recognition," *Sensors*, vol. 16, no. 1, p. 115, 2016.

[18] Android Developers. (2024) Motion sensors. [Online]. Available: https://developer.android.com/develop/sensors-and-location/sensors/sensors_motion

[19] J. Taborri, E. Palermo, S. Rossi, and P. Cappa, "Gait partitioning methods: A systematic review," *Sensors*, vol. 16, no. 1, p. 66, 2016.

[20] H. Xing, J. Li, B. Hou, Y. Zhang, and M. Guo, "Pedestrian stride length estimation from IMU measurements and ANN based algorithm," *Wireless Communications and Mobile Computing*, vol. 2017, p. 6091261, Feb. 2017.

[21] W. Zijlstra and A. L. Hof, "Assessment of spatio-temporal gait parameters from trunk accelerations during human walking," *Gait & Posture*, vol. 18, no. 2, pp. 1–10, 2003.

[22] C. Hall, A. Figueroa, B. Fernhall, and J. A. Kanaley, "Energy expenditure of walking and running: comparison with prediction equations," *Medicine and science in sports and exercise*, vol. 36, no. 12, pp. 2128–2134, 2004.

[23] Strava Support. (2026) Calorie calculation. [Online]. Available: https://support.strava.com/hc/en-us/articles/216917097-Calorie-Calculation

[24] Google Support. (2026) How google fit calculates calories. [Online]. Available: https://support.google.com/fit/answer/6090183

[25] Google Developers. (2026) Activity data types - calories expended. [Online]. Available: https://developers.google.com/fit/datatypes/activity

[26] Nike Inc. (2024) Can i use the NRC app to track indoor runs? [Online]. Available: https://www.nike.com/help/a/nrc-indoor-run

[27] Samsung Electronics. (2024) Samsung health developer documentation. [Online]. Available: https://developer.samsung.com/health

[28] J.-P. Choe and M. Kang, "Apple watch accuracy in monitoring health metrics: a systematic review and meta-analysis," *Physiological Measurement*, vol. 46, no. 4, p. 04TR01, Apr. 2025.

[29] F. Germini *et al.*, "Accuracy and acceptability of wrist-wearable activity-tracking devices: Systematic review of the literature," *JMIR mHealth and uHealth*, vol. 10, no. 1, p. e30791, Jan. 2022.

[30] SQLite Consortium. (2023) Sqlite home page. [Online]. Available: https://sqlite.org/

[31] Remi Rousselet. (2024) freezed – flutter package. [Online]. Available: https://pub.dev/packages/freezed

[32] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017.

[33] Strava Inc. (2024) Strava: Running, cycling and hiking app. [Online]. Available: https://www.strava.com/

[34] Google LLC. (2024) Google fit: Activity tracking and health data API. [Online]. Available: https://developers.google.com/fit

[35] M. Shoaib, S. Bosch, O. D. Incel, H. Scholten, and P. J. M. Havinga, "Complex human activity recognition using smartphone and wrist-worn motion sensors," *Sensors*, vol. 16, no. 4, p. 426, Mar. 2016.

[36] J. Perry, *Gait Analysis: Normal and Pathological Function*. SLACK Incorporated, 1992.

[37] A. Middleton, S. L. Fritz, and M. Lusardi, "Walking speed: The functional vital sign," *Journal of Aging and Physical Activity*, vol. 23, no. 3, pp. 314–322, 2015.

[38] A. R. Wu *et al.*, "Mechanics of very slow human walking," *Scientific Reports*, vol. 9, p. 18079, 2019.

[39] E. Angelats *et al.*, "Enhanced seamless indoor–outdoor tracking using time series of GNSS positioning errors," *ISPRS International Journal of Geo-Information*, vol. 13, no. 2, p. 72, 2024.

[40] Y. Zhu *et al.*, "A fast indoor/outdoor transition detection algorithm based on machine learning," *Sensors*, vol. 19, no. 4, p. 786, 2019.

[41] P. Tierney and N. Clarke, "A comparison of a smartphone app with other gps tracking type devices employed in football," *Exercise Medicine*, vol. 3, no. 4, pp. 1–6, 2019.

[42] M. T. Scott, T. J. Scott, and V. G. Kelly, "The validity and reliability of global positioning systems in team sport: a brief review," *The Journal of Strength & Conditioning Research*, vol. 30, no. 5, pp. 1470–1490, 2016.

[43] R. Gilgen-Ammann, T. Schweizer, and T. Wyss, "Accuracy of distance recordings in eight positioning-enabled sport watches: instrument validation study," *JMIR mHealth and uHealth*, vol. 8, no. 6, p. e17118, 2020.